



Replit App Description: Translation Management Dashboard for YouTube Videos

Project vision

I run multiple YouTube channels and translate other creators' videos into several languages. I currently track my work in multiple Google Sheets tables, which is time-consuming and error-prone. I need a **full-stack web application** that automates data entry and streamlines my workflow. The app should remove my dependency on Google Sheets, handle data automatically via APIs whenever possible, and provide a clean, professional interface for both desktop and mobile.

1. Core goals

1. **Analyze my current workflow** and translate it into intuitive app features. I insert video URLs, specify target languages, track translation statuses, archive completed translations, schedule future uploads, manage multiple channels, and review statistics. The new app should support all of these tasks and automate as much as possible.
 2. **Design and implement a modern, responsive web app** with a clean, intuitive UI for desktop and a carefully thought-out mobile layout. Provide both light and dark themes and allow users to switch between them.
 3. **Select an appropriate technology stack automatically**, leveraging Replit's environment. Use a modern full-stack framework (e.g., React with Next.js or similar) for the frontend, Node.js (or another suitable backend such as FastAPI) for the server, and integrate a relational database (PostgreSQL via Replit's managed Postgres or SQLite) to persist data. Use a component library (Material-UI, Tailwind, or similar) and responsive design to ensure the app looks great on both PCs and phones.
 4. **Integrate with external services** to automate work whenever possible. Provide configurable support for the YouTube API and Telegram Bot API. When API keys are present, fetch data automatically (e.g., video titles, publish status, view counts); when absent, allow manual data entry. Implement environment variables and secure secrets management for API keys.
 5. **Provide flexible settings and customization**, allowing me to adjust default languages, channels, notifications, and visual themes without editing code.
-

2. Feature breakdown

Below is an outline of the main pages and components you should build. Each page should be responsive and have desktop and mobile views. Use a card-based design where appropriate and keep information density manageable. Include search, sorting, and filtering for larger tables. All dates and times should respect the **Europe/Kyiv** time zone.

2.1 Translation queue (primary dashboard)

- Create a dashboard page listing videos that need translation, presented as **cards**. Each card should display the original video link (or thumbnail preview fetched via API), the languages selected for translation, and the status (not started / in progress / completed) for each language.
- **Adding videos:** Provide a form or quick-add input to paste a YouTube URL. After insertion, automatically fetch metadata (title, thumbnail) using the YouTube API if available, and display it

on the card. Below each video, show chips or buttons for each target language (populated from the default language settings). Allow the user to edit the list of target languages for that video.

- **Mark translation as complete:** When I click on a language chip, show a detailed form (e.g., modal or expandable section) to input the link of the translated video, select the channel where the translation will be published, specify the voice-over name/ID and gender used, and optionally schedule a future release date and time. If the release date/time is in the future, automatically create an entry in the **Scheduled Releases** page.
- Once all translations for a video are completed, automatically **move the video to the Archive** (see below) and remove it from the queue.

2.2 Archive / Completed translations

- Implement a page acting as an **archive** of all translated videos. Display a table or card list showing each original video link, the language translated, the link to the new translated video, the channel where it was published, the voice-over name and gender, and the publication date. Include search by original link and language to detect duplicate translations.
- Provide filters by language, channel, and date. This archive should help me avoid translating the same video twice into the same language.

2.3 Scheduled releases

- Create a page summarizing **future scheduled publications**. Each entry should show: original video, translation language, channel, voice-over used, and scheduled publish date/time. Sort by upcoming date. When less than 12 hours remain until publication, highlight the row or card with an accent color; when less than 2 hours remain, trigger an automatic **Telegram notification** (see notifications settings) and visually emphasize it.
- Include the ability to edit or reschedule a release date. Display the schedule in a calendar or list view with filters by channel and date range.

2.4 Default languages

- Provide a settings page where I can **set and modify the default list of target languages**. When adding new videos to the queue, these languages should be preselected. Allow adding or removing languages from this list at any time. If I remove a language, it should not affect existing cards but should no longer appear by default for new videos.

2.5 Channel management

- Implement a page to **manage YouTube channels**. I should be able to add, edit, and remove channels. For each channel, store:
 - Channel name and URL
 - Default language for published videos on that channel
 - Voice-over name or ID used on the channel
 - Voice-over gender (male/female)
 - The niche or topic of the channel (e.g., "science," "technology," etc.)
- Use these settings when filling out translation details (see the queue page) so I can quickly select the correct channel and voice without retyping.

2.6 Statistics page

- Build a **statistics dashboard**. Allow me to choose one or multiple channels and display key metrics such as:

- Total number of published translations across all channels or selected channels.
- Total videos per language.
- Number of scheduled releases.
- Any other logical criteria (e.g., number of translations by voice-over, translations per niche, etc.).
- Provide controls to show/hide specific metrics or data sets. Use simple charts (bar/line/pie) or cards to visualize the data. Integrate YouTube API to fetch view counts per channel if API keys are provided; otherwise, allow manual entry or display only internal statistics.

2.7 Action log / Event history

- Maintain an **activity log** of all significant events (video added, translation marked complete, video published, schedule changed, settings updated). Display this log in its own page with timestamps and descriptions. Allow filtering or toggling which events to track. This helps audit actions and maintain history.

2.8 Notifications

- Integrate with the **Telegram Bot API** to send notifications for important events, such as:
 - A scheduled release is less than 2 hours away.
 - A translation has been published.
 - System errors or issues.
- Provide a settings panel to customize which events trigger notifications and to edit the message templates. Support scheduled (time-based) notifications.

2.9 Global settings and customization

- Add a **global settings** page with the following sections:
 - **Theme and appearance:** choose light or dark theme, select accent colors, pick fonts, and set button styles.
 - **Layout builder:** provide a simple form builder or data-display configurator so I can choose which columns appear on each page and in what order. This can be implemented as a drag-and-drop interface.
 - **YouTube API integration:** provide fields to input API credentials and detailed inline help explaining how to obtain and use them. If keys are present, the system should fetch titles, thumbnails, publish status, and view statistics automatically. Otherwise, fallback to manual entry.
 - **Telegram API integration:** input bot token and chat ID. Provide instructions on how to create a bot and grant necessary permissions. Allow customizing the default notification messages and selecting which events trigger notifications.
 - **Scheduled tasks:** allow enabling/disabling time-based events (e.g., daily summaries) and customizing the timing.
 - **Other settings:** options like language of the UI (support Russian/English), timezone (default to Europe/Kyiv), and date format.

3. Technical requirements

1. **Framework and language:** Let the AI agent choose a modern full-stack framework that fits well in Replit's environment. A typical choice would be **React with Next.js** for the frontend and **Node.js/Express** or **FastAPI** for the backend. Use TypeScript for better type safety if available. The app should follow best practices (MVC or component-based architecture, separation of concerns). Use Replit's managed Postgres or SQLite for the database, with an ORM (e.g., Prisma

or SQLAlchemy) to manage models such as Videos, Translations, Channels, Languages, Schedules, Logs, Settings, etc.

2. **Responsive design:** Implement responsive layouts using CSS frameworks or component libraries such as **Material-UI**, **Tailwind CSS**, or **Ant Design**. Provide distinct, well-thought-out layouts for desktop and mobile. Use media queries to rearrange elements and ensure that cards and tables scale gracefully on smaller screens.
 3. **API integrations:** Implement modules to interact with the YouTube API (data retrieval, upload scheduling) and the Telegram Bot API (sending messages). Use environment variables for API keys and secrets. Provide fallback paths (manual input) when APIs are not configured.
 4. **Scheduling and background tasks:** Use a scheduling library (e.g., **node-cron** or Python's **APScheduler**) to handle timed events such as sending Telegram notifications 2 hours before a scheduled release and 12-hour warnings within the app. Ensure scheduled tasks respect the configured timezone.
 5. **Authentication (optional):** If needed, implement a simple authentication system so that only authorized users (e.g., me) can access the dashboard. Otherwise, the app can be single-user with no login.
 6. **Testing and validation:** Use Replit's **Agent testing capabilities** (Agent 3 can test apps in a browser and fix issues automatically ¹). Validate user input, handle API errors gracefully, and ensure the UI remains responsive.
 7. **Documentation:** Throughout development, document code and configuration files so future changes are easier. Add inline comments and a README summarizing how to deploy and maintain the app within Replit. Provide guidance on obtaining API keys and linking Telegram bots.
-

4. Design and UX guidelines

- Use a **clean, modern aesthetic**. Opt for a professional look with plenty of white space (or dark backgrounds in dark mode), clear typography, and intuitive iconography.
 - Provide **consistent navigation** (e.g., a sidebar or tab bar) to switch between the translation queue, archive, scheduled releases, statistics, channels, languages, logs, settings, and add-video pages.
 - Make interactive elements (buttons, chips, cards) large enough to tap on mobile. Use collapsible menus or drawers on smaller screens.
 - Include contextual tooltips and help markers to explain unfamiliar fields, especially in the settings pages where API configurations occur.
 - Use color coding to communicate status (e.g., green for completed translations, yellow/orange for upcoming scheduled releases within 12 hours, red for releases within 2 hours). Provide accessible color palettes for both themes.
-

5. Example workflow summary (for reference)

1. **Add Video:** User pastes an original video link on the "Add Video" page. The app fetches metadata and displays a card in the queue with default languages selected.
2. **Translation Process:** For each language on the card, the user clicks the language chip, enters the translated video URL, selects the publication channel (pre-populated from channel settings), chooses voice-over details, and optionally schedules the publish date/time. Upon saving, the language status updates to "Completed" and, if scheduled, the release is added to the Scheduled Releases page.

3. **Complete Video:** When all languages for a video are completed, the card is automatically moved to the Archive page. The user can search the archive to avoid duplicates in the future.
 4. **Scheduled Notification:** The app monitors upcoming releases. If a release is within 12 hours, the card in Scheduled Releases is highlighted. At 2 hours before publication, a Telegram message is sent automatically.
 5. **Statistics and Logs:** At any time, the user can view the Statistics page to see aggregated data and the Logs page to review actions.
-

6. Deliverables

- Generate the full project in Replit based on this specification. Once built, I should be able to run the app and access all pages listed above.
- Provide a polished desktop and mobile UI, with both light and dark themes.
- The system should be ready to accept YouTube and Telegram API keys, work offline without them, and allow comprehensive customization via the settings pages.

Please use this description as the basis for the Replit Agent's feature list and implementation plan. If any requirements are unclear or incomplete, ask clarifying questions before proceeding. Remember to leverage Replit's AI capabilities: start with a design prototype if helpful, then build full functionality ². The goal is a professional, user-friendly translation management dashboard that eliminates manual spreadsheet work.

¹ Replit — Introducing Agent 3: Our Most Autonomous Agent Yet
<https://blog.replit.com/introducing-agent-3-our-most-autonomous-agent-yet>

² Replit Docs
<https://docs.replit.com/replitai/agent>